CSC1013 Introduction to R

Group Assignment

AI Usage Documentation

Project: sapply(data, pls_clean_me)

Dataset: Climate Change & Global Warming

Name:	John Bernard Basañes, Kacper Szymoniak
Student ID:	70835, 94352, 62750, 60279
Programme:	DS1
Module Code:	CSC1013
Project Name:	sapply(data, pls_clean_me)
Group Members:	Atharv Soni, Alvaro Lopez
Submission Date:	5th April 2026

Generative AI (Gen AI) Declaration

(a) I hereby certify that the Generative AI tools used, and the purpose for which they were used, are declared below using the relevant template.

Task 1 – Environment Setup

Declaration Table

Requirement	Reference
Project & Question	Project 1; Task 1 – Environment Setup
Owner / Contributors	John Bernard Basañes – Task Lead / Setup; Kacper Szymoniak, Atharv Soni, Alvaro Lopez
Original Working Before AI Use	See below
Tools	Google Gemini 2.5 Pro – DCU student version. 22/03/2026
Significant Prompts	See below
Final AI Output	See below
Critical Reflection	See below

Original Work Before AI Use

Before consulting Gemini, our initial setup attempt was a single flat script with repeated library() calls and no installation handling:

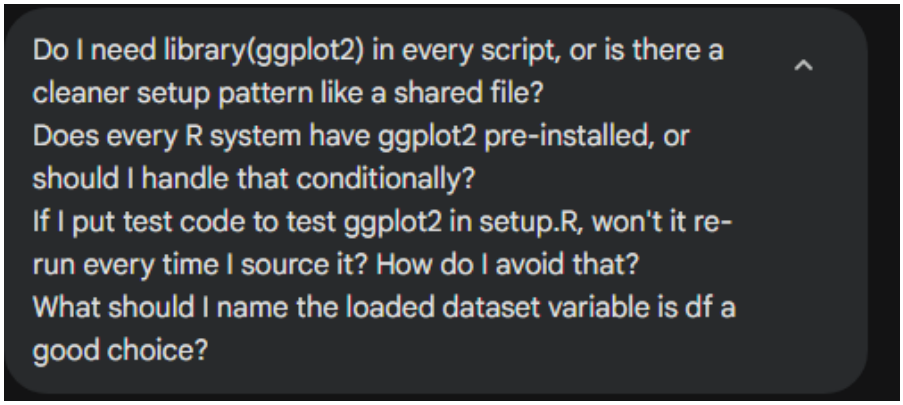
```
# Task 1 - initial attempt (before AI consultation)
library(ggplot2)
library(dplyr)

df <- read.csv("Beginner_Climate_Change_Dataset_20_Features_1200_Rows.csv")
```

Issues with this approach:

- library() would need to be repeated at the top of every script
- No handling for machines where ggplot2 is not installed
- df shadows the built-in R function stats::df()
- Test code mixed in with setup, so it would re-run on every source()

Significant Prompt



The prompt sent to Gemini covered four questions in a single message:

- Do I need library(ggplot2) in every script, or is there a cleaner shared setup pattern?
- Does every R system have ggplot2 pre-installed, or should installation be handled conditionally?
- If test code is placed in setup.R, won't it re-run every time the file is sourced? How do I avoid this?
- Is df a good variable name for a loaded dataset?

Final AI Output

Managing your environment efficiently is one of those "quality of life" upgrades that separates a messy project from a professional one. Here is the breakdown on how to keep your scripts clean and your setup robust.

---

1. The "Shared File" Pattern

You don't need `library(ggplot2)` at the top of every single script if you use a **configuration script**. This is a standard practice in larger projects.

- **The Pattern:** Create a file named `init.R` or `setup.R` that contains all your `library()` calls, theme settings, and global variables.
- **The Usage:** At the start of your work scripts, simply use `source("setup.R")`.

  Pro Tip: While this is cleaner, if you plan on sharing individual scripts with others, they might get frustrated if the script doesn't work because it's missing a dependency hidden in another file. For standalone scripts, keep the `library()` calls visible.

---

2. Handling Installation

No, `ggplot2` is **not** pre-installed in R. Only "Base R" packages (like `stats` or `graphics`) come out of the box.

To handle this gracefully without manually checking every time, use the `require()` logic or a package manager like `pacman`.

The "Clean" Conditional Way:

```R
if (!require("ggplot2", quietly = TRUE)) {
    install.packages("ggplot2")
    library(ggplot2)
}
```

The "Professional" Way:
Use the `pacman` package. It checks if a package is installed; if not, it downloads it, and then it loads it—all in one line.

```R
if (!require("pacman")) install.packages("pacman")
pacman::p_load(ggplot2, dplyr, tidyr)
```

---

3. Avoiding Re-running Test Code

If you put `plot(mtcars)` in your `setup.R`, it will indeed pop up every time you source it. To avoid this, use a **logical gate**.

The "Run-Once" Guard:

```R
# setup.R
library(ggplot2)

# Only run tests if explicitly asked
run_tests <- FALSE

if (run_tests) {
    print("Testing ggplot2...")
    print(ggplot(mtcars, aes(wt, mpg)) + geom_point())
}
```

Alternatively, keep your setup (libraries/functions) in `setup.R` and your experiments in a separate `scratchpad.R`.

---

4. Naming: Is `df` a good choice?

In the R community, `df` is the "standard" shorthand for a data frame, but it has a significant downside: `df` **is also a built-in R function** (the Density function for the F-distribution).

The Verdict:

- **For quick tutorials/reproducible examples:** `df` is fine. Everyone knows what it means.
- **For real projects: Avoid it.** It's better to use something descriptive or a slightly modified shorthand.

Choice	Evaluation	Why?
`df`	C-	Collides with `stats::df()`. Not descriptive.
`data`	D	Too generic; also a built-in function name.
`my_data`	B	Safe, but a bit "beginner" sounding.
`raw_sales`	A+	Tells you exactly what is inside.
`stn_df`	B+	(Station Data Frame) - Combines description with type.

Export to Sheets

Would you like me to draft a "gold standard" `setup.R` template that you can copy-paste into your current project?

Summary of key points from Gemini's output:

- Recommended the source("setup.R") shared file pattern to avoid repeating library() calls
- Suggested if (!require("ggplot2", quietly = TRUE)) for conditional installation
- Proposed a run_tests <- FALSE logical gate to prevent test code re-running on source()
- Also suggested the pacman package as a more concise alternative
- Advised against df as a variable name due to collision with stats::df()

Critical Reflection

What worked well:

Gemini's explanation of the source() pattern was immediately applicable. Splitting setup into Task_1.1_Setup.R and sourcing it from Task_1.2_Setup.R kept the test script clean. The variable naming advice was also useful, renaming df to climate is more descriptive and removes the ambiguity with the built-in function.

What was challenging:

Gemini suggested two installation approaches (if(!require()) and pacman) without clearly recommending one. This required us to make a judgment call about which was appropriate for a student project.

Changes made and why:

- Adopted source() pattern — directly useful, no issues.
- Used if (!require()) logic — standard R, requires no extra packages.
- Did NOT use pacman — beyond module material and introduces an undeclared dependency.
- Did NOT use the run_tests flag — test plots placed in a separate script instead, which is cleaner.
- Renamed df to climate — more descriptive, avoids the naming collision Gemini flagged.

Actionable knowledge for the future:

Always check whether an AI suggestion introduces a new package dependency before adopting it. AI output should be a starting point for understanding, not a copy-paste solution.

Task 7 – Scatter Plot with Linear Regression Line

Declaration Table

Requirement	Reference
Project & Question	Project 1; Task 7 – Scatter Plot Visualisation
Owner / Contributors	Kacper Szymoniak – Task Lead / Setup; John Bernard Basañes Atharv Soni, Alvaro Lopez
Original Working Before AI Use	See below
Tools	Google Gemini 2.5 Pro – DCU student version. 24/03/2026
Significant Prompts	See below
Final AI Output	See below
Critical Reflection	See below

Original Work Before AI Use

A working scatter plot had already been written for Task 7. The plot showed CO2 concentration vs temperature anomaly but only displayed the raw data points. We wanted to add a linear regression line to show the overall trend more clearly:

```
source("scripts/Task_1.1_Setup.R")
climate_temperature_differences <-
readRDS("data/climate_temperature_differences.rds")

aggregated_data <- climate %>%
  group_by(year) %>%
  summarize(
    co2_concentration_mean = mean(co2_concentration_ppm),
    temperature_mean       = mean(temperature_anomaly)
  )

scatter_plot <- ggplot(aggregated_data, aes(x = co2_concentration_mean, y =
temperature_mean)) +
  geom_point(color = "steelblue", size = 2) +
  labs(
    title = "CO2 Concentration (ppm) vs Average Temperature Anomaly",
    x     = "Average CO2 Concentration (ppm)",
    y     = "Average Temperature Anomaly"
  ) +
  theme_minimal()

print(scatter_plot)
ggsave("charts/Task_7_Scatter_plot.png", plot = scatter_plot)
```

The issue: the plot only showed individual points with no trend line. We asked Gemini to explain geom_smooth() and show customisation options.

Significant Prompt

```
I have this ggplot2 scatter plot code for my Task 7:
R

source("scripts/Task_1.1_Setup.R")climate_temperature_
differences <-
readRDS("data/climate_temperature_differences.rds")ag
gregated_data <- climate %>% group_by(year) %>%
summarize( co2_concentration_mean =
mean(co2_concentration_ppm), temperature_mean =
mean(temperature_anomaly) )scatter_plot <-
ggplot(aggregated_data, aes(x =
co2_concentration_mean, y = temperature_mean)) +
geom_point(color = "steelblue", size = 2) + labs( title =
"CO2 Concentration (ppm) vs Average Temperature
Anomaly", x = "Average CO2 Concentration (ppm)", y =
"Average Temperature Anomaly" ) +
theme_minimal()print(scatter_plot)ggsave("charts/Task_
7_Scatter_plot.png", plot = scatter_plot)
Right now it only has the points. I want to add a linear
regression line to it.
Please explain in simple terms:

What does the linear regression line do in this context?
What is it supposed to show?
What should it look like on the plot?
What is the basic command to add a straight linear
regression line using geom_smooth()?
Then show me improved versions with useful
customization options, such as:Removing the confidence
interval band
Changing the color and thickness of the line
Making the confidence band lighter or changing its color
Adding the regression equation or R² value on the plot (if
possible)
Any other common useful options
Give me the full updated scatter_plot code each time so
I can just copy-paste and test it.
Thanks!
```

The prompt asked Gemini to:

- Explain in simple terms what a linear regression line does in this context and what it is supposed to show
- Describe what it should look like on the plot
- Show the basic geom_smooth() command to add a straight linear regression line
- Show improved versions with: removing the confidence interval band, changing line colour and thickness, making the confidence band lighter, and adding the R^2 value if possible
- Provide the full updated scatter_plot code each time

Final AI Output

Adding a linear regression line is a great way to make sense of the relationship between CO_2 and temperature. It turns a "cloud of points" into a clear story.

1. What does the line actually do?

In this context, the linear regression line acts as a **"line of best fit."** It calculates the mathematical middle ground between all your data points.

- **What it shows:** It highlights the **trend**. While individual years might fluctuate, the line shows the overall direction and strength of the relationship.
- **What it looks like:** Since there is a strong positive correlation between CO_2 and warming, you should see a straight line sloping upward from the bottom-left to the top-right.

---

2. The Basic Command

The magic function in `ggplot2` is `geom_smooth()`. To make it a straight line, you must specify the method as `"lm"` (Linear Model).

Basic Code:

R

```
scatter_plot <- ggplot(aggregated_data, aes(x = co2_concentration_mean, y = tempe
    geom_point(color = "steelblue", size = 2) +
    geom_smooth(method = "lm") + # Adds the basic line with a grey confidence rib
    labs(
        title = "CO2 Concentration (ppm) vs Average Temperature Anomaly",
        x = "Average CO2 Concentration (ppm)",
        y = "Average Temperature Anomaly"
    ) +
    theme_minimal()

print(scatter_plot)
```

---

3. Improved Versions with Customization

A. Clean Line (No Confidence Band)

By default, `ggplot2` adds a grey "ribbon" (the standard error). If you want just the line for a cleaner look, use `se = FALSE`.

```r
scatter_plot <- ggplot(aggregated_data, aes(x = co2_concentration_mean, y = tempe
    geom_point(color = "steelblue", size = 2) +
    geom_smooth(method = "lm", se = FALSE, color = "firebrick", linewidth = 1.5)
    labs(
        title = "CO2 Concentration vs Temperature Trend",
        x = "Average CO2 Concentration (ppm)",
        y = "Average Temperature Anomaly"
    ) +
    theme_minimal()

print(scatter_plot)
```

B. Customizing the Confidence Band

If you like the band but find the default grey too heavy, you can change its color and transparency (`alpha`).

```r
scatter_plot <- ggplot(aggregated_data, aes(x = co2_concentration_mean, y = tempe
    geom_point(color = "steelblue", size = 2) +
    # fill changes the band color, alpha makes it lighter (0 is invisible, 1 is s
    geom_smooth(method = "lm", color = "darkblue", fill = "lightblue", alpha = 0.
    labs(
        title = "CO2 Concentration vs Temperature (with Confidence Interval)",
        x = "Average CO2 Concentration (ppm)",
        y = "Average Temperature Anomaly"
    ) +
    theme_minimal()

print(scatter_plot)
```

C. Adding the Regression Equation and R^2

To display the math on the plot, the easiest way is to use the `ggpubr` library. It handles the calculation and placement for you. (Note: You may need to run `install.packages("ggpubr")` first).

```r
library(ggpubr)

scatter_plot <- ggplot(aggregated_data, aes(x = co2_concentration_mean, y = tempe
    geom_point(color = "steelblue", alpha = 0.6) +
    geom_smooth(method = "lm", color = "black") +
    # Adds the equation and R-squared value automatically
    stat_regline_equation(label.y = 1.2, aes(label = ..eq.label..)) +
    stat_regline_equation(label.y = 1.1, aes(label = ..rr.label..)) +
    labs(
        title = "Statistical Relationship: CO2 vs Temperature",
        x = "Average CO2 Concentration (ppm)",
        y = "Average Temperature Anomaly"
    ) +
    theme_minimal()

print(scatter_plot)
```

Summary of Useful Options in `geom_smooth()`

Argument	What it does	Example
`method = "lm"`	Forces the line to be a straight linear regression.	`method = "lm"`
`se`	Shows/hides the shaded confidence interval.	`se = FALSE`
`color`	Changes the color of the line itself.	`color = "red"`
`fill`	Changes the color of the shaded confidence band.	`fill = "grey"`
`alpha`	Sets the transparency of the shaded band.	`alpha = 0.3`
`linewidth`	Sets the thickness of the line.	`linewidth = 2`
`linetype`	Changes the style of the line (e.g., dashed).	`linetype = "dashed"`

Summary of key points from Gemini's output:

- Explained that geom_smooth(method = "lm") fits a straight line using a linear model, showing the overall trend rather than individual fluctuations
- Noted that a positive slope is expected given the known relationship between CO2 and temperature
- Showed that se = FALSE removes the grey confidence ribbon for a cleaner look
- Demonstrated color, linewidth, fill, and alpha as useful customisation arguments
- Suggested the ggpubr package and stat_regline_equation() to display the R^2 value on the plot
- Provided a summary table of all geom_smooth() options with examples

Critical Reflection

What worked well:

The explanation of method = "lm" was clear and useful, understanding that "lm" stands for linear model helped us connect the ggplot2 syntax to the statistical concept covered in lectures. The customisation options for se, color, and linewidth were all straightforward to apply directly to our existing code.

What was challenging:

Gemini suggested using the ggpubr package to display the R^2 value. While useful, this is an additional package not covered in the module, and the stat_regline_equation() syntax using the deprecated ..eq.label.. notation also produced a warning in our R version.

Changes made and why:

- Added geom_smooth(method = "lm", se = FALSE, color = "firebrick", linewidth = 1) — the clean line without a confidence band was chosen as it keeps the chart readable.
- Did NOT use ggpubr or stat_regline_equation() — beyond module scope and produced deprecation warnings. The trend is visually clear without the equation.
- Kept geom_point() colour as steelblue to maintain contrast with the red regression line.

- The final code is an improvement on both the original (no trend line) and the AI output (removed the ggpubr dependency).

Actionable knowledge for the future:

When Gemini suggests a package outside the module, check whether the same result can be achieved with base ggplot2 first. In this case, the regression line itself was sufficient without needing to print the equation on the chart.